Leapfrog Assignment 2: A Retrieval Augmented Generation (RAG) Pipeline for LF Job Descriptions

Raag Yatchu Maharjan

Leapfrog Technology Inc., KTM

Abstract

This report documents the design, implementation, and evaluation methodology of a Retrieval-Augmented Generation system built for natural-language job search over a structured CSV dataset of 1,000 job listings. The system takes a user query, retrieves semantically relevant job description chunks from a locally persisted ChromaDB vector store, and passes those chunks as grounded context to a local language model that produces a cited, concise answer. The ingestion pipeline cleans HTML job descriptions, splits them into overlapping chunks, and embeds them using the BAAI/bge-small-en-v1.5 sentence transformer. At query time, a hybrid retriever combines dense vector search with keyword scoring, fuses both result sets through Reciprocal Rank Fusion, and reranks candidates with a cross-encoder before prompt construction. The language model backend is Hugging Face Transformers running Qwen2.5-3B-Instruct locally. The system is served through a FastAPI application with Pydantic request validation and structured JSON logging. This report aims to cover the full system architecture, each component in the data flow, prompt design decisions, engineering considerations, evaluation metrics, known limitations, and recommended future improvements.

***K*eywords:** *Retrieval-Augmented Generation, Vector Search, ChromaDB, Hugging Face Transformers, FastAPI, Pydantic, Reranker*

1 Introduction

1.1 What is Retrieval-Augmented Generation (RAG)?

RAG or Retrieval-Augmented Generation is an architectural paradigm in natural language processing that combines information retrieval with text generation. A standalone language model answers from its learned internal parameters alone, which means that it relies on knowledge encoded dyring the training phase plus the context provided by a user prompt during that time. This can lead to real practical issues, it can hallucinate information, fabricate plausibl souding details when its parametric memeory is insufficient. Standards language models are also incapable of fitting a large dataset into every prompt.

A RAG system addressses these issues by retrieving relevant external context at query time and injecting the context into the model prompt. The model is then instructed to answer only from the retrieved context.

For the context of this project, the external knowledge base is a CSV of job listings provided by Leapfrog Technology Inc. The retrieval layer finds matching job description chunks, the generation layer uses those chunks to produce a job recommendation based on the user query.

1.2 Purpose of the System

The job RAG Pipeline loads jobs from a CSV file, cleans and normalizes the HTML formatted job descriptions, converts them into searchable text chunks, embeds those chunks into vector representations, stores the vectors in a local ChromaDB vector database, and injects the language model with context at query time for a more grounded response.

1.3 Objectives

My objectives of this assignment are to:

1. Design and implement a RAG pipeline that can effectively retrieve relevant job descriptions based on user queries.
2. Evaluate the performance of the retrieval and generation components using appropriate metrics.
3. Document the entire process, including the methodology, results, and insights gained from the project.
4. Optionally try to implement a reranker model to further improve the retrieval performance of the system.
5. Identify limitations of the current system and propose potential improvements for future iterations.

2 Methodology

The provided dataset LFJobs.csv consists of 1,000 job listings with the following columns:

- **ID:** A unique identifier for each job listing.
- **Job Category:** The category or field of the job.
- **Job Title:** The title of the job position.
- **Company Name:** The name of the company offering the job.
- **Publication Date:** The date when the job listing was published.
- **Job Location:** The location where the job is based.
- **Job Level:** The seniority level of the job (e.g., entry-level, mid-level, senior).
- **Tags:** Keywords or tags associated with the job listing.

- **Job Description:** A detailed description of the job responsibilities, requirements, and other relevant information.

The "Job Description" column contains HTML formatted text, which requires cleaning and normalization before it can be used for retrieval and generation. Normalization involves removing HTML tags, special characters, and extra whitespace, as well as converting the text to lowercase for consistency.

The entire methodology of the project can be broadly categorized into 5 main phases:

(a) **EDA and Data Ingestion:** Cleaning data, building documents, chunking, embedding, and storing to ChromaDB.
(b) **Retrieval:** Using a hybrid retriever to find relevant job description chunks based on user queries, combine the results using reciprocal rank fusion.
(c) **Reranking:** Reranking the retrieved chunks using a cross-encoder to improve the relevance of the context provided to the language model.
(d) **Generation:** Constructing prompts for the LLM as a wrapper around the retrieved context and generating responses to user queries.
(e) **Serving:** Building a FastAPI application to serve the RAG system and handle user queries in real-time.
(f) **Output and Evaluation:** Evaluating the performance of the retrieval and generation components using appropriate metrics and documenting the results.

The above methodology is depicted in the process flowchart below:

2.1 EDA and Data Ingestion Phase

Exploratory Data Analysis:

EDA (Exploratory Data Analysis) refers to the initial phase of data analysis where we explore the dataset to understand its structure, identify patterns, and detect anomalies.

In the context of this project, EDA of the provided dataset involved examining the distribution of features, checking for missing values, and visualizing relationships between features and the target variable (CLASS).

From the preliminary analysis, it was observed that the dataset had a significant class imbalance, with a higher proportion of diabetic patients compared to non-diabetic and pre-diabetic patients.

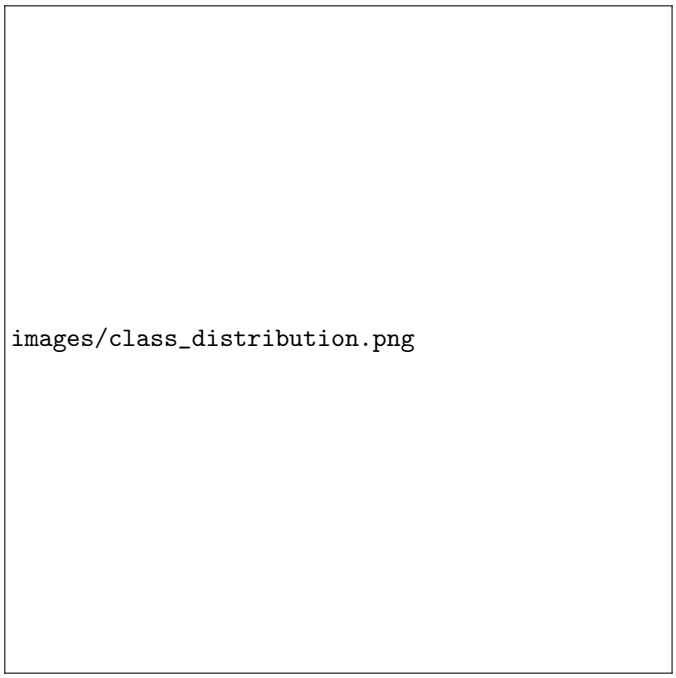

Figure 1: Class Distribution in the Dataset

Also looking at the entire data profile report obtained from Sweetviz, I found that 2 columns (Gender and CLASS) had dirty entries (non-standard entries) and white spaces that needed to be cleaned up.

Figure 2: Dirty Entries in Gender Column

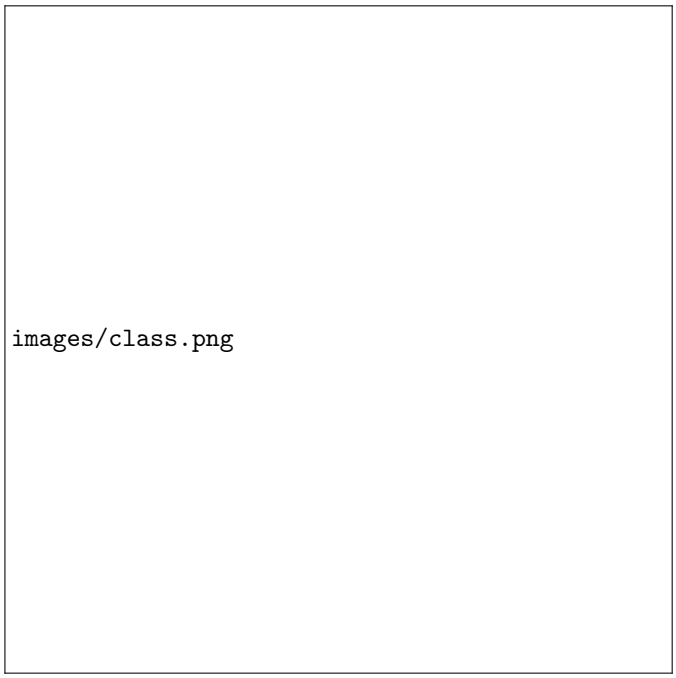

Figure 3: Dirty Entries in CLASS Column

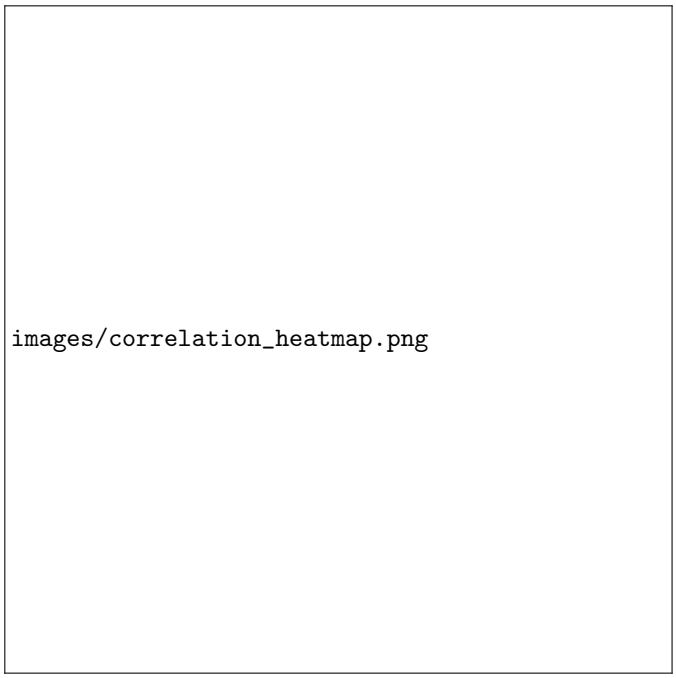

Figure 4: Correlation Matrix of the Features

I also computed the correlation matrix of the features to understand how they relate to each other and to the target variable. The corelation matrix revealed that Urea and Cr had a strong positive correlation, which could be useful for feature selection and model training.

Other moderately correlated features included BMI and Age, which are known risk factors for diabetes. After identifying these relationships, I proceeded to pairplot and scatter plot them to futher investigate the nature of their relationships.

Scatter plotting the strongly correlated features i.e. Urea and Cr with a regression line revealed a clear positive linear reltionship between them. This suggests that they may be capturing similar information about the patients' kidney function, which is relevant for diabetes prediction.

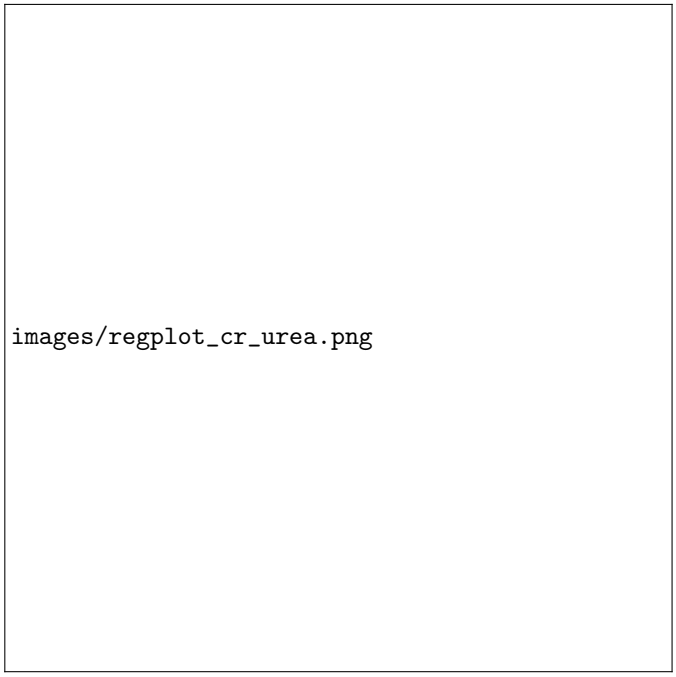

Figure 5: Scatter Plot of Urea vs Cr with Regression Line

Moving on to the moderately correlated features, I plotted a pairplot of all the features with CLASS as

the hue to visualize how they relate to the target variable.

The pairplot revealed that BMI and HbA1c had some separation between the classes, while other features showed more overlap.

This tells us that BMI and HbA1c could be important features for classification, while the others may require more complex modeling to capture their relationships with the target variable.

After the end of EDA the information on the dataset which I managed to obtain can be summarized as follows:

Data Characteristics:

- The dataset contains 1000 records with 13 features and a target variable (CLASS) that has three classes: N, P, and Y.
- Heavily imbalanced dataset with more diabetic patients (Y) than non-diabetic (N) and pre-diabetic (P) patients.
- Non-linear patterns and very frequent overlap between classes in the feature space, suggesting that simple linear models may not perform well.
- Numerous outliers present throughout the features.
- Non-consolidated and dirty entries in the AGE and Gender columns.
- ID and Patient_ID columns are irrelevant to the analysis and contain noise.

Feature Relationships:

- HbA1c appeared to be the most separating feature between the classes, followed by BMI.
- Gathered meaningful relationships between features with most with moderately positive correlation, and a few with strong positive correlation (Urea and Cr).

From the above information, insights I could gather from the EDA phase which are necessary for the next steps are as follows:

Insights:

- **Class Imbalance**: Entails the need for weighted evaluation and stratified splitting.
- **Non-linear Relationships**: Suggests the need for non-linear models and robust preprocessing (binning).
- **Outliers**: Necessitates the use of robust imputation and scaling techniques.
- **Dirty Entries**: Requires data cleaning and standardization.
- **Irrelevant Features**: Calls for feature selection and removal of ID columns.

Data Preprocessing:

After the EDA phase, I proceeded to preprocess the data based on the insights obtained. The preprocessing steps I applied to the dataset are as follows:

- **Data Cleaning**: Cleaned and consolidated the entries in the CLASS and Gender.
- **Feature Selection**: Removed the ID and Patient_ID columns as they were irrelevant to the analysis and introduced noise.
- **Binning**: Binned the AGE and BMI features into groups to capture non-linear relationships and reduce the impact of outliers.
- **Imputation and Scaling**: Applied median imputation and standardization to numeric features.
- **Encoding**: One-hot encoded the categorical features for a consistent representation.

After preprocessing, the dataset was ready for model training and evaluation. The cleaned and processed data was then split into training and testing sets using stratified sampling to maintain the class distribution.

2.2 Model Development Process

The preprocessed data was split into 3 sets, the training set, the validation set, and the test set. The training set was used to train the models, the validation set was used for model validation, and the test set was used for final evaluation. The splits were in the ratio of 7:1:2 for training, validation, and testing respectively.

Model Building Phase:

For the scope of this project, I implemented a classification and clustering pipeline as I was instructed.

Classification models are a type of supervised learning algorithm that are used to predict the class labels of new instances based on the patterns learned from the training data. Clustering models are a type of unsupervised learning algorithm that are used to group similar instances together based on their features without using class labels.

Supervised Models:

The supervised models I implemented for classification are:

(a) Random Forest Classifier
(b) XGBoost Classifier
(c) Support Vector Machine (SVM) Classifier

The supervised models needed to be trained on splits of the data so as to prevent data leakage and allow for proper evaluation. The training characteristic curves of the models are as follows:

Figure 7: Training Curve of the Random Forest Classifier

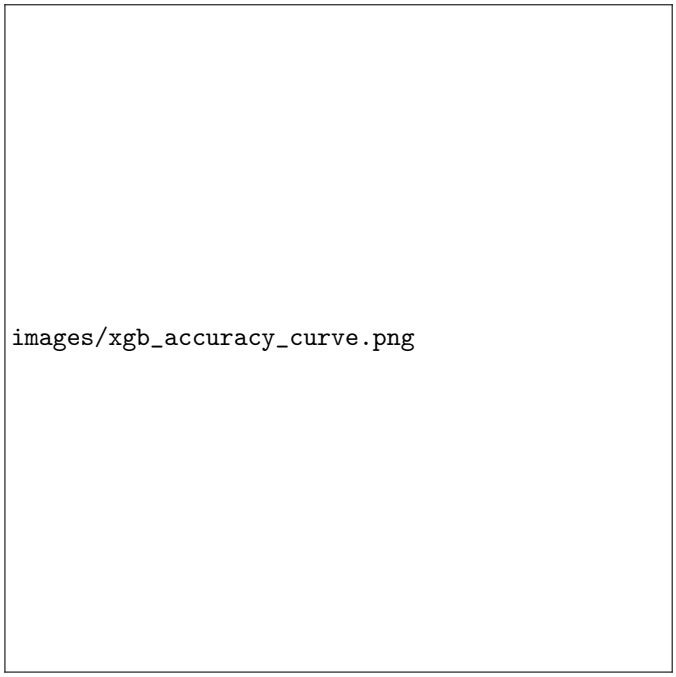

Figure 8: Training Curve of the XGBoost Classifier

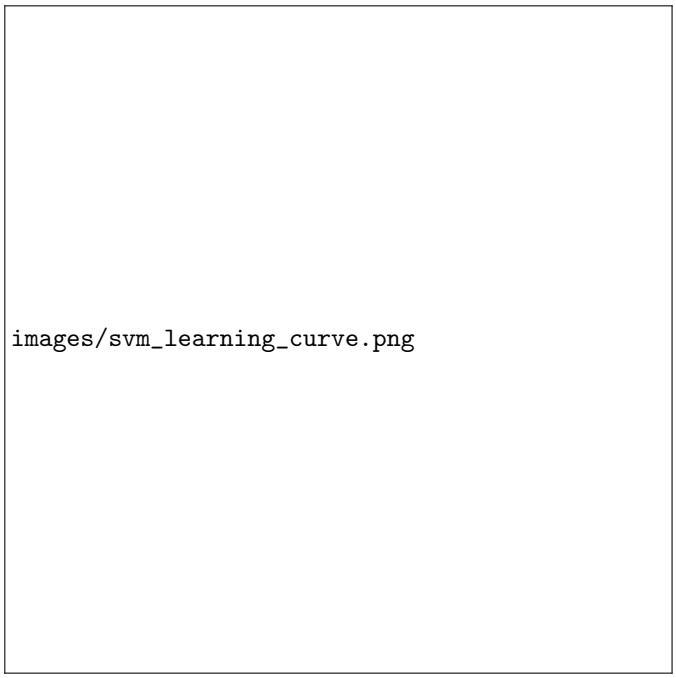

Figure 9: Training Curve of the SVM Classifier

From the above training curves, we can see that all three models are learning and improving their performance on the training data over time. The validation curve is also catching up with the training curve, which suggests that the models are not overfitting and are generalizing well to unseen data.

After training the models, their weights were saved to disk for a reproducible evaluation without the need to re-run training. This also allows for easy comparison of the models' performance on the test set.

Feature importance plots were also generated for the Random Forest and XGBoost models to understand which features were most influential in the classification process. The feature importance plots are as follows:

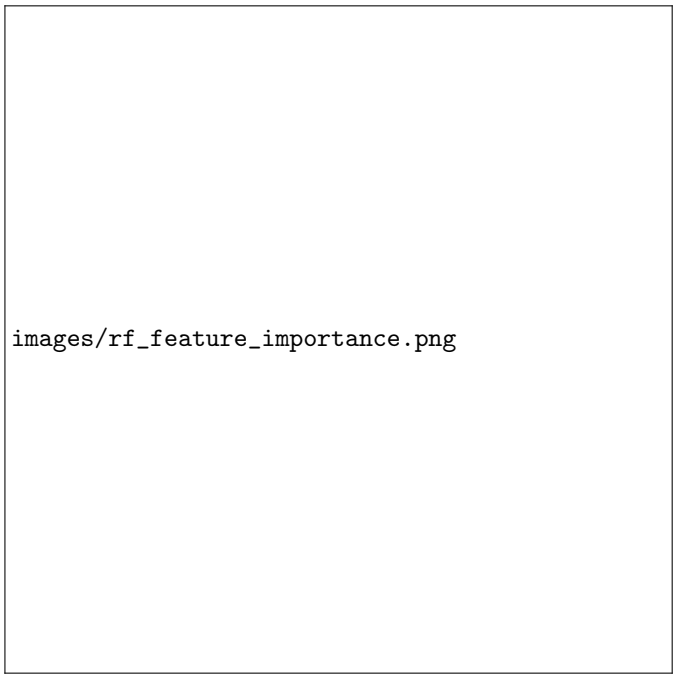

Figure 10: Feature Importance of the Random Forest Classifier

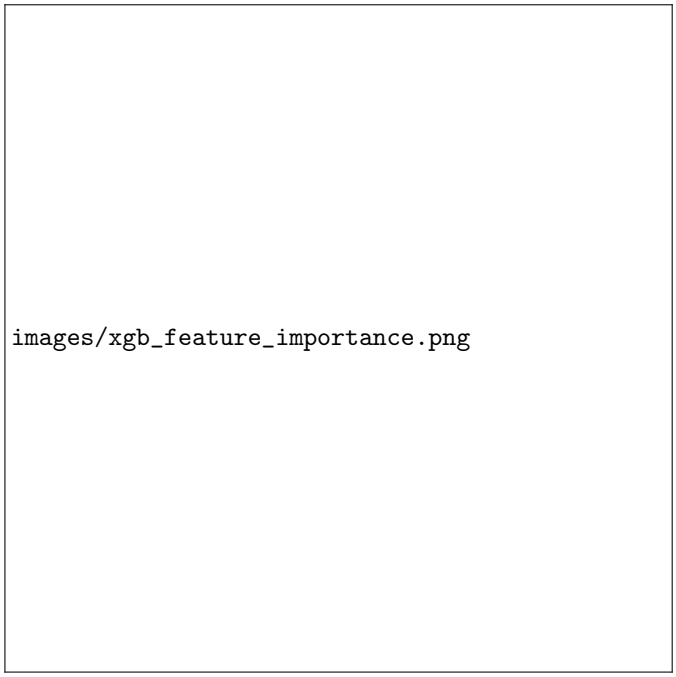

Figure 11: Feature Importance of the XGBoost Classifier

As we predicted in the EDA phase, HbA1c and BMI are among the most important features for both models, which aligns with our understanding of diabetes risk factors.

AGE binning also helped to capture the non-linear relationship between age and diabetes risk, as the binned AGE feature also appears in the top important features for both models.

SVM does not provide feature importance in the same way as tree-based models as it is a linear model.

Unsupervised Models:

The unsupervised models I implemented for clustering are:

(a) K-Means Clustering
(b) Agglomerative Clustering
(c) Gaussian Mixture Model (GMM)

The unsupervised models were trained on the full dataset rather than the held-out test split since they do not use class labels and are meant to discover hidden structures in the data. PCA was applied to reduce the dimensionality of the data to two dimensions for scatter plot visualization, making it possible to see how well the unsupervised structure maps onto the actual class boundaries.

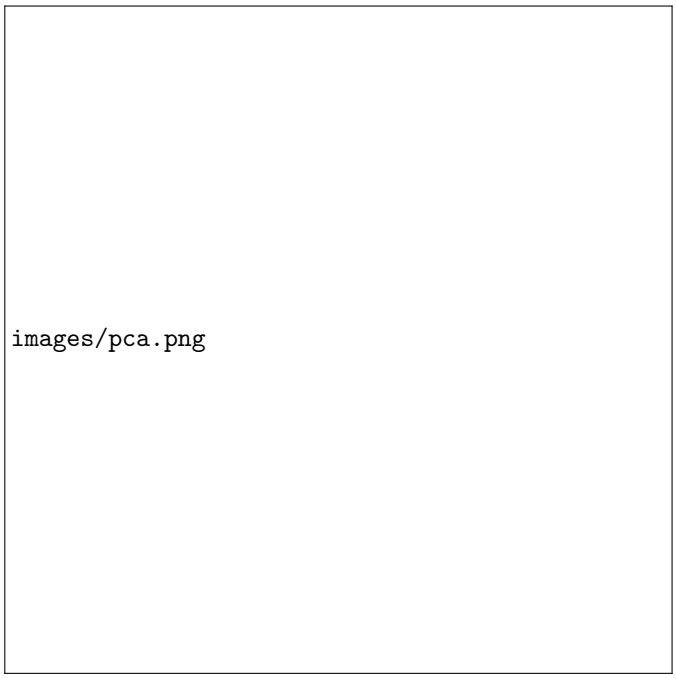

Figure 12: PCA Visualization of Clusters

From the PCA visualization, we can see that there is some separation between the classes, but there is also a lot of overlap, which suggests that the clusters may not perfectly align with the true class labels.

For the K-Means model, I also plotted an elbow curve to select the optimal number of clusters (k) based on the within-cluster sum of squares (WCSS).

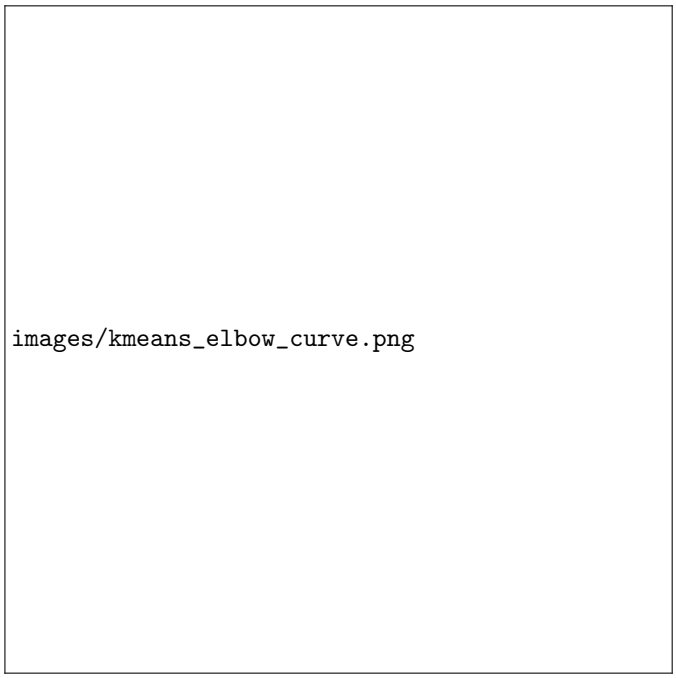

Figure 13: Elbow Curve for K-Means Clustering

But as we can see from the elbow curve, there is no clear elbow point, which suggests that there may not be a well-defined number of clusters in the data. Hence, I used K-value of 3 for K-Means clustering to match the number of classes in the target variable.

For the Agglomerative Clustering and GMM, I also set the number of clusters to 3 for consistency and comparison with K-Means.

The clusters produced by each model are as follows:

Figure 14: Clusters produced by K-Means Clustering

Figure 15: Clusters produced by Agglomerative Clustering

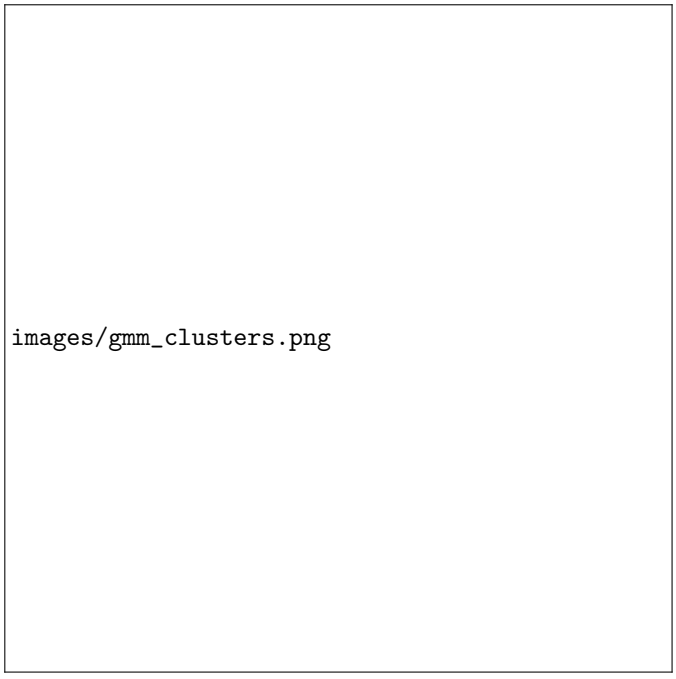

Figure 16: Clusters produced by Gaussian Mixture Model

Each of the clustering models produce a similar clustering structure, with some overlap between the clusters, which is consistent with the PCA visualization and the nature of the data.

We can also see an oblong shape along the first principal component, which suggests that there may be some underlying structure in the data that is not captured by the original features.

There also seems to be some outliers in the data which do not clearly belong to any of the clusters. This is a limitation of the clustering models, as they may struggle to assign these outliers to a cluster, which can affect the overall performance and interpretation of the clusters.

Each of the clustering models were also saved to disk for reproducibility and further analysis without the need to re-run training.

2.3 Model Evaluation Process

After traning the supervised and unsupervised models, I proceeded to evaluate their performance using standard metrics. There are various different metrics for evaluating classification and clustering models, and I used a combination of them to get a comprehensive understanding of the models' performance.

Evaluation of Supervised Models:

The supervised models were evaluated using the following metrics:

1. Accuracy:

The proportion of correct predictions out of the total preditctions made. Mathematically,

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{i}$$

2. Precision:

The proportion of true positive predictions out of all positive predictions made. Mathematically,

$$Precision = \frac{TP}{TP + FP} \tag{ii}$$

3. Recall:

The proportion of true positive predictions out of all actual positive instances. Mathematically,

$$Recall = \frac{TP}{TP + FN} \tag{iii}$$

4. F1 Score:

The harmonic mean of precision and recall, providing a balance between the two. Mathematically,

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{iv}$$

5. ROC-AUC:

The area under the receiver operating characteristic curve, which plots the true positive rate against the false positive rate at various threshold settings. A higher AUC indicates better model performance.

$$AUC = \int_0^1 TPR(FPR)\, dFPR \tag{v}$$

Where,
TP denotes true positives. TN denotes true negatives. FP denotes false positives. FN denotes false negatives.

All of which were obtained from the confusion matrix of the models' predictions on the test set, and were calculated for each class as well as in a weighted and macro-averaged manner to account for class imbalance. The confusion matrix for each model are as follows:

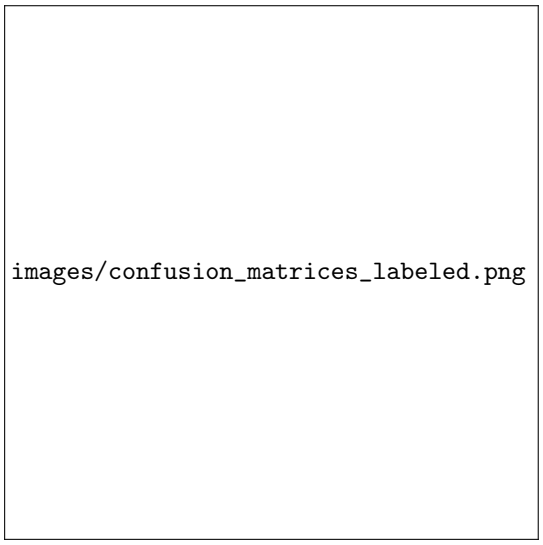

Figure 17: Confusion Matrices of the Supervised Models

The evaluation results of the supervised models are tabulated below:

Table 1: Per-class precision, recall, and F1-score.

Class	Precision	Recall	F1-score
Random Forest			
N	1.00	1.00	1.00
P	1.00	0.90	0.95
Y	0.99	1.00	1.00
XGBoost			
N	1.00	0.90	0.95
P	1.00	0.90	0.95
Y	0.98	1.00	0.99
SVM			
N	0.79	0.90	0.84
P	0.58	0.70	0.64
Y	0.99	0.96	0.97

As predicted during the EDA phase, the Random Forest and XGBoost models performed very well with high precision, recall, and F1-scores across all classes, while the SVM model struggled with the pre-diabetic class (P) due to the non-linear relationships and class imbalance in the data.

Linear models like SVM may not capture the complex patterns in the data as effectively as tree-based models, which can handle non-linear relationships and interactions between features. So this is consistent with our understanding of the data and the nature of the models.

The ROC-AUC (Micro-Average) scores for each class and model are as follows:

Table 2: Micro-average ROC-AUC for each supervised model.

Model	Micro-average AUC
Random Forest	0.999
XGBoost	0.999
SVM	0.995

Figure **??** shows a bar chart comparing the evaluation metrics of the three supervised models, which clearly illustrates the superior performance of the Random Forest and XGBoost models compared to SVM, especially in terms of precision, recall, and F1-score.

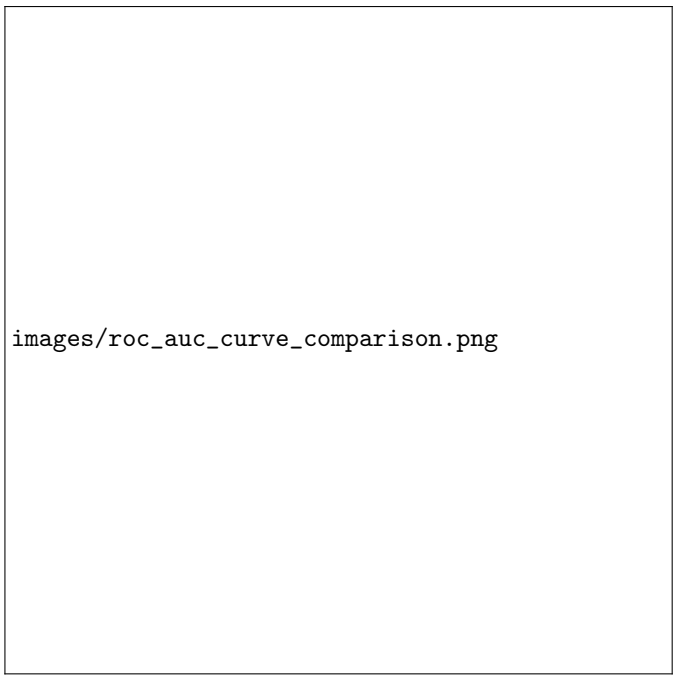

Figure 19: ROC Curves for the Supervised Models

All 3 models achieved very high ROC-AUC scores, i.e. close to 1, which indicates that they are performing well in distinguishing between the classes. Values closer to 0.5 would indicate that the model is performing no better than random guessing.

2.3.1 Evaluation of Unsupervised Models:

Evaluation of unsupervised models is more different than supervised models since we do not have true class labels to compare against. There is no dedicated metric to evaluate the accuracy of clustering models, but we can usea combination of internal and external metrics to assess the quality of the clusters. The unsupervised models were evaluated using the following metrics:

1. Silhouette Score:

Measures how similar an instance is to its own cluster compared to other clusters. A higher silhouette score indicates better-defined clusters. Mathematically,

$$Silhouette = \frac{b - a}{\max(a, b)} \tag{vi}$$

Where:
a : the average distance between an instance and all other instances in the same cluster (intra-cluster distance).
b : the average distance between an instance and all other instances in the nearest cluster (inter-cluster distance).

2. Davies-Bouldin Index:

Measures the average similarity between clusters. A lower Davies-Bouldin index indicates better clustering. Mathematically,

$$DB = \frac{1}{k} \sum_{i=1}^{k} \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d(c_i, c_j)} \right) \tag{vii}$$

Where:
k : the number of clusters.
σ_i : the average distance between instances in cluster i and the centroid of cluster i.
$d(c_i, c_j)$: the distance between the centroids of clusters i and j.

3. Calinski-Harabasz Score:

Measures the ratio of between-cluster variance to within-cluster variance. A higher Calinski-Harabasz score indicates better clustering. Mathematically,

$$CH = \frac{Tr(B_k)}{Tr(W_k)} \times \frac{n - k}{k - 1} \tag{viii}$$

Where:
B_k : the between-cluster dispersion matrix.
W_k : the within-cluster dispersion matrix.
n : the total number of instances.
k : the number of clusters.

4. Adjusted Rand Index (ARI):

Measures the similarity between the true class labels and the cluster assignments, adjusted for chance. A higher ARI indicates better clustering. Mathematically,

$$ARI = \frac{RI - ExpectedRI}{\max(RI) - ExpectedRI} \tag{ix}$$

Where:
RI : the Rand Index, which measures the agreement between two clusterings.
ExpectedRI : the expected value of the Rand Index under random labeling.

5. Normalized Mutual Information (NMI):

Measures the mutual information between the true class labels and the cluster assignments, normalized to account for chance and the number of clusters. A higher NMI indicates better clustering. Mathematically,

$$NMI = \frac{I(X; Y)}{\sqrt{H(X)H(Y)}} \tag{x}$$

Where:
$I(X; Y)$: the mutual information between the true labels X and the cluster assignments Y.
$H(X)$: the entropy of the true labels.
$H(Y)$: the entropy of the cluster assignments.

The evaluation results of the unsupervised models are tabulated below:

Table 3: Clustering evaluation metrics for the unsupervised models.

Metric	K-Means	Hierarchical	Gaussian Mixture
Silhouette Score	0.221910	0.222706	0.216481
Davies-Bouldin Score	1.378506	1.515881	2.598974
Calinski-Harabasz Score	157.718387	144.809497	122.731468
Adjusted Rand Index	0.587506	0.512008	0.417449
Normalized Mutual Information	0.447570	0.412447	0.370465

We can see from the evaluation metrics tabulated in Table **??** that the K-Means model performed the best among the three unsupervised models, with the highest silhouette score, lowest Davies-Bouldin score, highest Calinski-Harabasz score, and highest ARI and NMI scores.

The Agglomerative Clustering model performed slightly worse than K-Means, while the Gaussian Mixture Model performed the worst among the three, which is consistent with the PCA visualization and the nature of the data.

As Gaussian Mixture Models assume that the data is generated from a mixture of Gaussian distributions, which may not be the case for this dataset, leading to poorer performance compared to K-Means and Agglomerative Clustering. K-means performed best because it is a simple and effective clustering algorithm that works well in general clustering scenarios.

3 Result Analysis and Conclusion

3.1 Result Analysis

The results of the supervised models indicate that the Random Forest and XGBoost classifiers performed exceptionally well in predicting diabetes classes, with high precision, recall, and F1-scores across all classes.

The SVM classifier, on the other hand, struggled with the pre-diabetic class (P) due to the non-linear relationships and class imbalance in the data, which is consistent with our understanding of the data and the nature of the models.

The ROC-AUC scores for all three models were very high, indicating that they are performing well in distinguishing between the classes.

The results of the unsupervised models indicate that the K-Means clustering algorithm performed the best among the three unsupervised models, with the highest silhouette score, lowest Davies-Bouldin score, highest Calinski-Harabasz score, and highest ARI and NMI scores.

The Agglomerative Clustering model performed slightly worse than K-Means, while the Gaussian Mixture Model performed the worst among the three, which is consistent with the PCA visualization and the nature of the data. The Gaussian Mixture Model's assumption of data being generated from a mixture of Gaussian distributions may not hold for this dataset, leading to poorer performance compared to K-Means and Agglomerative Clustering.

Overall, the results of this project demonstrate the importance of choosing appropriate models and evaluation metrics based on the characteristics of the data and the problem at hand.

3.2 Conclusion

In conclusion, this project provided a comprehensive analysis of the diabetes dataset using both supervised and unsupervised machine learning techniques. The supervised models, particularly Random Forest and XGBoost, performed exceptionally well in predicting diabetes classes, while the SVM model struggled with the pre-diabetic class due to the non-linear relationships and class imbalance in the data. The unsupervised models revealed some underlying structure in the data, with K-Means performing the best among the three clustering algorithms. The project also highlighted the importance of thorough exploratory data analysis, proper data preprocessing, and careful model selection and evaluation to achieve good performance and meaningful insights from the data. Hence, the objectives of the project were successfully achieved, and the results provide valuable insights into the diabetes dataset and the effectiveness of different machine learning techniques for classification and clustering tasks.

4 Limitations and Future Enhancements

4.1 Limitations

- The dataset used in this project is relatively small, which may limit the generalizability of the results and the performance of the models.
- The class imbalance in the dataset may have affected the performance of the models, particularly the SVM classifier, which struggled with the pre-diabetic class.
- The unsupervised models may not have captured the true underlying structure
- Clustering is inherently subjective and may not perfectly align with the true class labels, which can affect the interpretation of the clusters.
- Selecting optimal cluster parameters (e.g., number of clusters) can be challenging and may require more sophisticated techniques than the elbow method.
- Comparing the performance of supervised and unsupervised models can be difficult since they are designed for different tasks and may not be directly comparable.

4.2 Future Enhancements

- Hyperparameter tuning and optimization for the supervised models to further improve their performance.
- Exploring additional supervised models such as neural networks or ensemble methods to see if

they can capture the complex relationships in the data better than the current models.

- Handling class imbalance using techniques such as SMOTE (Synthetic Minority Over-sampling Technique) or class weighting to improve the performance of models on the minority classes.
- Additional feature engineering and selection techniques to identify the most relevant features for classification and clustering.
- Lipid profile ratios with the help of an expert could be engineered as additional features to capture more complex relationships in the data.

5 APPENDIX

5.1 Project Directory Structure

```
project/
|
|-- classificationBook.ipynb
|-- clusteringBook.ipynb
|-- dataPrep.ipynb
|-- edaBook.ipynb
|-- evalClassModel.ipynb
|-- evalClusteringModel.ipynb
|
|-- dataset/
|-- images/
|-- models/
|-- processed_arrays/
|
‘-- requirements.txt
```

5.2 Technologies Used

The following tools and libraries were used in this project:

Tools:

- Git (version control)
- PyCharm (IDE)
- Miniconda (environment and package management)

Python Libraries:

- `pandas`
- `numpy`
- `matplotlib`
- `seaborn`
- `scikit-learn`
- `xgboost`
- `lightgbm`
- `catboost`
- `plotly`
- `ipywidgets`
- `sweetviz`
- `notebook`
- `jupyterlab`
- `ipykernel`
- `setuptools==68.2.2`
- `torch`
- `torchvision`
- `torchaudio`
- `cupy-cuda12x`
- `joblib`

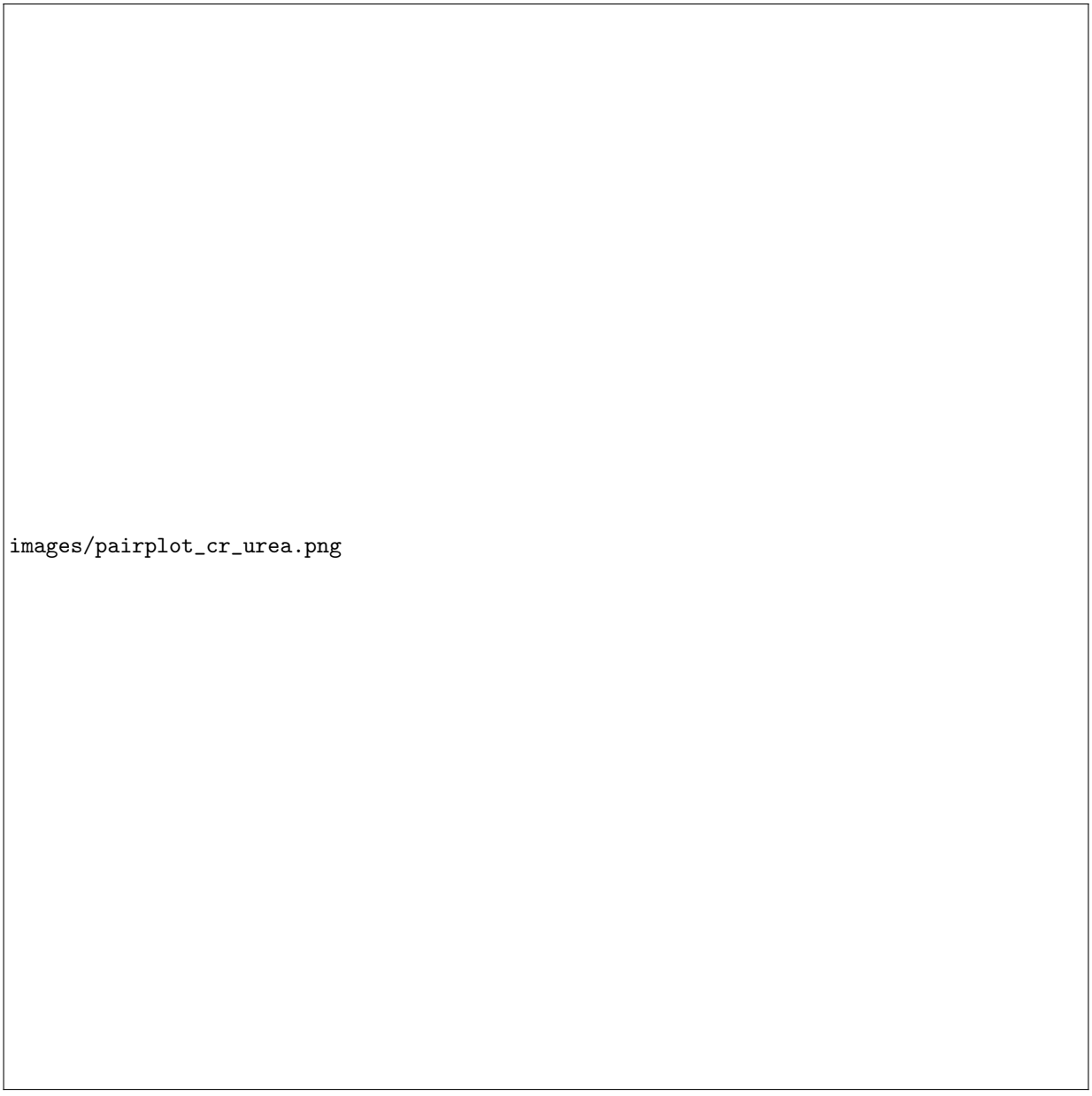

Figure 6: Pairplot of Features with CLASS as Hue

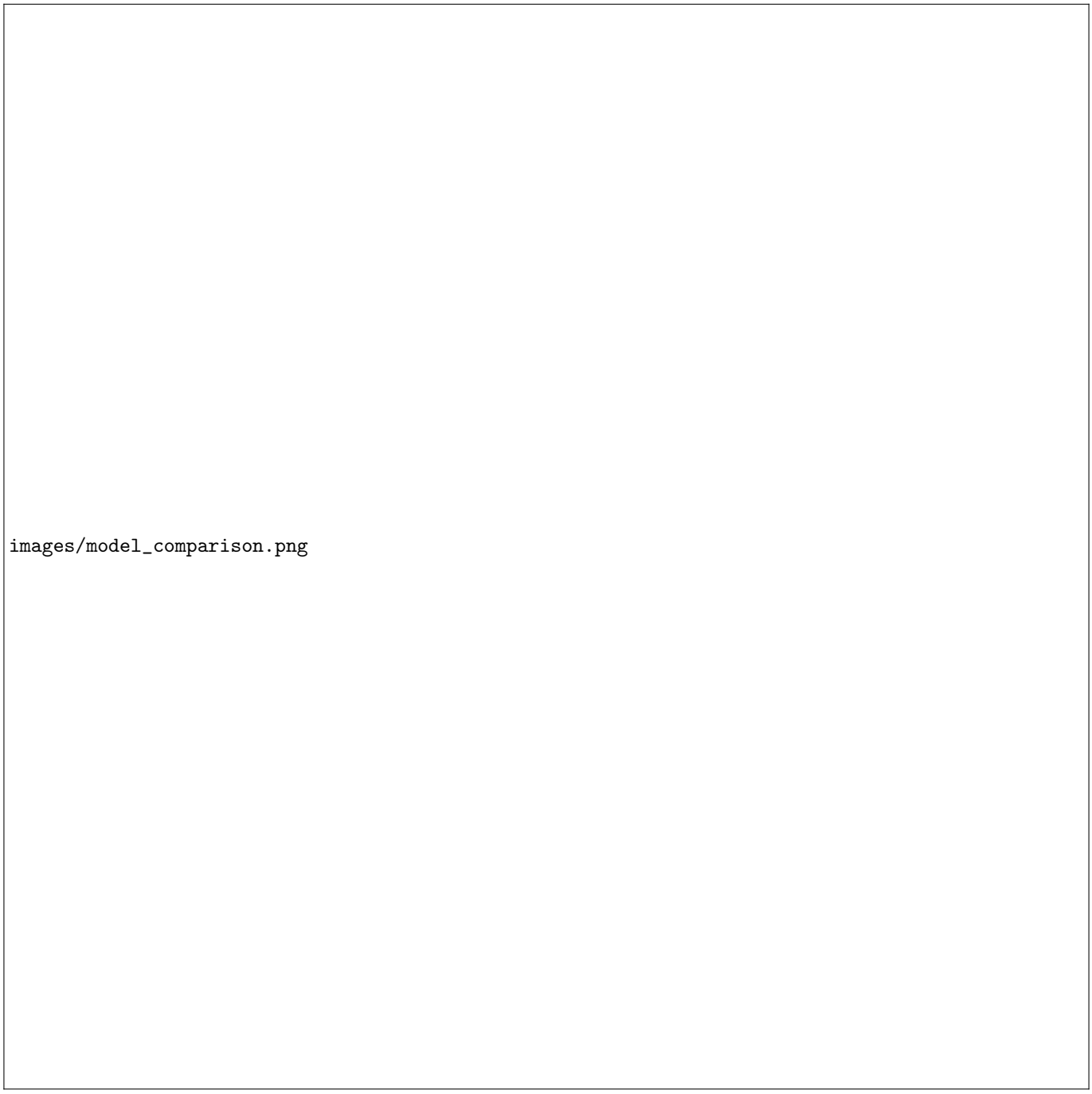

Figure 18: Bar Charts Comparing the Evaluation Metrics of the Supervised Models

images/clustering_model_comparison.png

Figure 20: Bar Charts Comparing the Evaluation Metrics of the Unsupervised Models

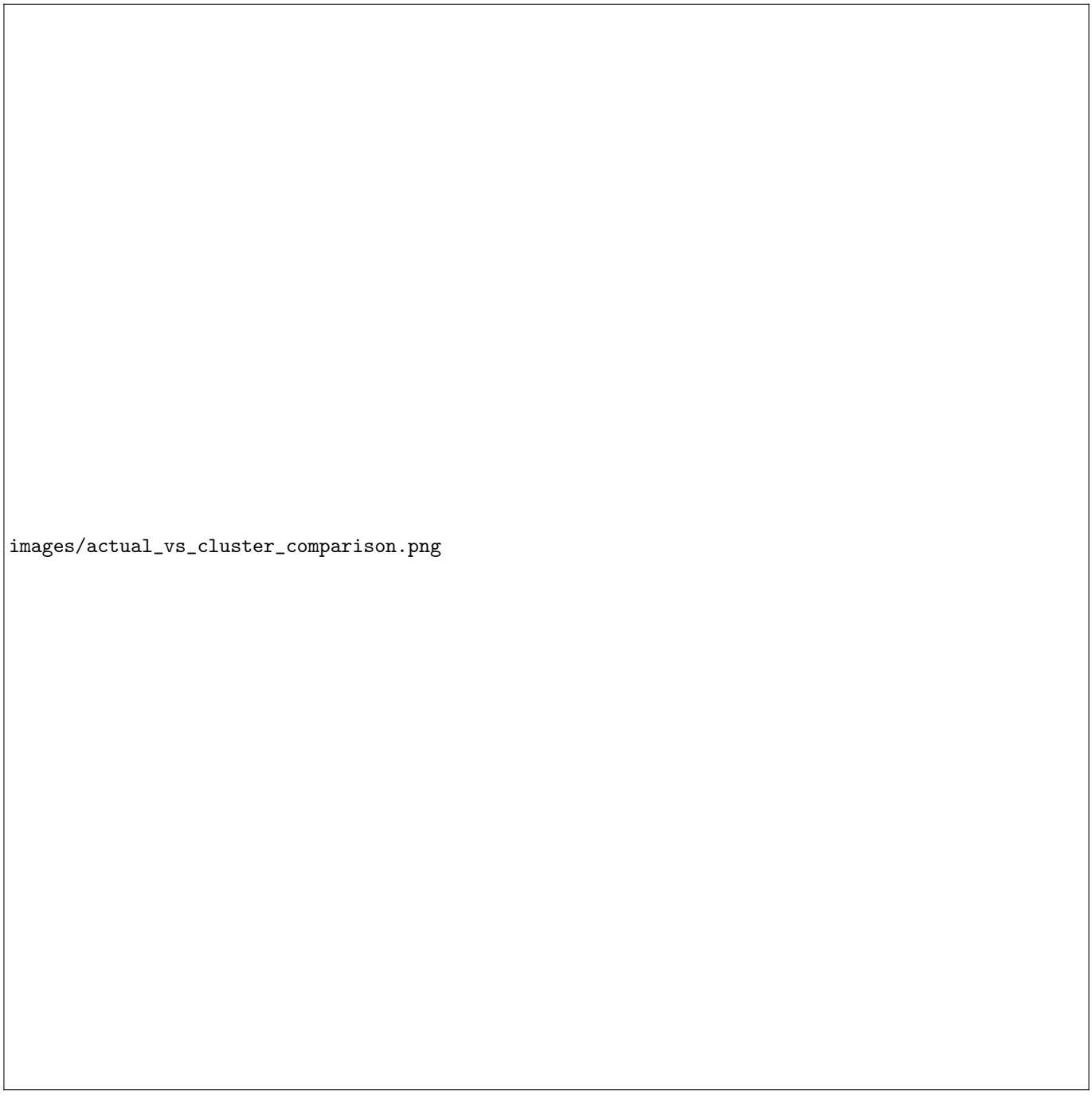

Figure 21: PCA Visualization of Clusters for Each Unsupervised Model with Actual Class Labels as Hue